Cache Behavior Lab Analysis

Measurements below were taken on a 12th Gen Intel Core i5-12600K (20 MB L3) with MinGW-W64 GCC 13.2.0. `clock()` on this toolchain reports whole milliseconds, so a run under 1 ms prints as 0.00 ms and the program omits the ratio. Both traversals produced the same sum at every size (49,500,000; 198,000,000; 792,000,000; 3,168,000,000), so the timing gap is the access order, not a different result.

Section 1: Performance Measurements

Command: `gcc cache_lab.c -o cache_lab` (no optimization flags, which is `-O0`) followed by `./cache_lab`.

Matrix size	Row-major (ms)	Column-major (ms)	Ratio (col/row)
1000 x 1000	2.00	2.00	1.00x
2000 x 2000	6.00	12.00	2.00x
4000 x 4000	30.00	67.00	2.23x
8000 x 8000	99.00	274.00	2.77x

At 1000 x 1000 both methods land on the same 2 ms tick, so the ratio looks flat. From 2000 upward, column-major takes longer, and the ratio grows with the matrix. On the 8000 x 8000 matrix, column-major takes 274 ms and row-major takes 99 ms, a 2.77x difference for the identical sum.

Section 2: Optimization Level Comparison

Matrix size	-O0 row	-O0 col	-O0 ratio	-O2 row	-O2 col	-O2 ratio	-O3 row	-O3 col	-O3 ratio
1000 x 1000	2.00	2.00	1.00x	0.00	1.00	—	0.00	1.00	—
2000 x 2000	6.00	12.00	2.00x	2.00	5.00	2.50x	1.00	5.00	5.00x
4000 x 4000	24.00	61.00	2.54x	5.00	52.00	10.40x	4.00	51.00	12.75x
8000 x 8000	94.00	273.00	2.90x	20.00	256.00	12.80x	13.00	268.00	20.62x

`-O3` does not eliminate the gap. On the 8000 x 8000 matrix the column-major time stays near 270 ms from `-O0` through `-O3`, while the row-major time falls from 94 ms to 13 ms, so the ratio grows from about 2.9x to about 20.6x. GCC vectorizes the contiguous row walk with 16-byte loads and leaves the column walk as a scalar gather of one int per row; it does not interchange the loops. Faster code makes the

memory pattern show up more clearly, because the row loop stops being limited by extra instructions and the column loop is still waiting on its loads.

Section 3: Assembly Analysis

Both functions were compiled on Compiler Explorer with x86-64 gcc 13.2 and `-O2`, the same compiler that produced the timings.

- Row-major: https://godbolt.org/z/ncqTxKG6h
- Column-major: https://godbolt.org/z/n9P87PGq3

`traverse_row_major` at `-O2`

```
1   traverse_row_major:
2           movsx   rax, DWORD PTR current_size[rip]
3           test    eax, eax
4           jle     .L5
5           mov     rdi, QWORD PTR matrix[rip]
6           lea     r8, [0+rax*4]
7           xor     edx, edx
8           lea     r9, [rdi+rax*8]
9   .L3:
10          mov     rax, QWORD PTR [rdi]
11          lea     rsi, [r8+rax]
12  .L4:
13          movsx   rcx, DWORD PTR [rax]
14          add     rax, 4
15          add     rdx, rcx
16          cmp     rsi, rax
17          jne     .L4
18          add     rdi, 8
19          cmp     rdi, r9
20          jne     .L3
21          mov     rax, rdx
22          ret
23  .L5:
24          xor     edx, edx
25          mov     rax, rdx
26          ret
27  current_size:
28          .zero   4
29  matrix:
30          .zero   8
```

Row-major assembly from Compiler Explorer, x86-64 gcc 13.2 -O2

The inner loop (`.L4`) loads one int, adds 4 to the data pointer, accumulates, and compares against the end of the row. The row pointer is loaded once per row (`.L3`), then reused for the whole row.

`traverse_column_major` at `-O2`

```
traverse_column_major:
        movsx   rax, DWORD PTR current_size[rip]
        test    eax, eax
        jle     .L5
        mov     r8, QWORD PTR matrix[rip]
        lea     r9, [0+rax*4]
        xor     esi, esi
        xor     edx, edx
        lea     rdi, [r8+rax*8]
.L3:
        mov     rax, r8
.L4:
        mov     rcx, QWORD PTR [rax]
        add     rax, 8
        movsx   rcx, DWORD PTR [rcx+rsi]
        add     rdx, rcx
        cmp     rdi, rax
        jne     .L4
        add     rsi, 4
        cmp     r9, rsi
        jne     .L3
        mov     rax, rdx
        ret
.L5:
        xor     edx, edx
        mov     rax, rdx
        ret
current_size:
        .zero   4
matrix:
        .zero   8
```

Column-major assembly from Compiler Explorer, x86-64 gcc 13.2 -O2

The inner loop (`.L4`) loads a row pointer, adds 8 to move to the next pointer, then loads the int at `[row + column]`. The column offset (`rsi`) advances by 4 only after every row has been visited.

The two listings are both short scalar loops, on the order of five or six instructions in the inner loop, so one function is not doing a much larger amount of arithmetic. The addressing is different: row-major strides through consecutive ints, and column-major reloads a row pointer on every element and indexes that row by the current column. That is enough to explain a large timing gap. The instructions

themselves are cheap. The addresses they generate determine whether each load hits the line the processor just filled. The same conclusion shows up at `-O3` in the local build: the row loop becomes a `movdqu` of 16 bytes plus `paddq` accumulation, and the column loop stays the scalar gather above. Similar instruction streams with different address order produce very different cache behavior, which is where the performance difference originates.

Section 4: Cache Behavior Explanation

C stores a true multidimensional array in row-major order. For `int a[R][C]`, element `a[i][j]` is at `base + (i * C + j) * sizeof(int)`. Consecutive columns of one row sit in consecutive addresses, and the same column of the next row is `C` integers away. This lab follows that rule inside each row and adds one indirection. `allocate_matrix` stores an array of pointers and `malloc`s every row separately, so `matrix[i][j]` and `matrix[i][j + 1]` are adjacent 4-byte ints, while `matrix[i + 1][j]` is a different heap block. A cache line on this x86-64 CPU is 64 bytes, or 16 ints. When `traverse_row_major` reads `matrix[i][j]`, the processor fills a line with that int and the next fifteen columns of the same row. The inner loop consumes those neighbors immediately, so they hit in L1. The address stream is a constant stride of 4 bytes, which the hardware prefetcher recognizes, and the row pointer is loaded once per row. That is the `.L4` loop in the `-O2` listing, which does `add rax, 4` after each load. At `-O3` those loads widen to 16 bytes, and the 8000-row time falls to 13 ms.

`traverse_column_major` keeps `j` fixed and advances `i`, so the next address is `matrix[i + 1][j]`. The line just filled still holds later columns of the current row, and this iteration does not use them. Each step loads the next row pointer and then one int at a column offset. Separate row allocations put successive iterations on different pages, so the L1 cache and the TLB see a new line, and often a new page, on every element. Those neighbors are used only on a later column, after every other row has been touched. On the 8000 matrix the column time stays near 270 ms through `-O3`.

GCC does not interchange these loops. The matrix is an `int **`, and the `-O2` listings keep the source order: a data pointer stepping by 4, versus a pointer-table cursor stepping by 8 with a reload of `[row + column]` on every iteration. At `-O3` the vectorizer widens the contiguous row walk and leaves the column walk scalar. The compiler can make a contiguous walk faster. It cannot supply spatial locality the source order does not already have, so the same sum still differs by more than 10x on the largest matrix.